

Applied Science Private University
Faculty of Information Technology



Project Phase #2
Parallel Programming Course – 1301421
Second Semester 2025/2026
Total Marks: 15

Submission Deadline: 19/05/2026 at 11:59 PM

Instructor(s):

Dr. Mansour Alhelalat

Dr. Ayham Alomari

Engr. Nada Abdalfattah

Phase Overview: The objective of this phase is to apply sequential and parallel programming concepts using Java by processing large datasets and analyzing program performance.

Students are required to implement:

- Sequential file processing
- Parallel file processing using outer threading
- Parallel file processing using inner threading
- Execution time analysis
- Performance comparison using different numbers of threads

This phase focuses on performance evaluation, scalability, and understanding the effect of parallelism on execution time.

Requirements:

1. Dataset Reading:

Each group must use the dataset selected in Phase #1.

The dataset may contain: **.txt, .csv, or .xlsx files**. Students must implement a Java program capable of reading and processing all files correctly.

2. Sequential Processing:

Students must implement two types of sequential processing:

A. Simple Processing

Examples include:

- Counting records
- Finding maximum or minimum values
- Counting categories or keywords

The execution time must be computed and recorded.

B. Complex Processing

Examples include:

- Floating-point arithmetic operations
- Multi-condition filtering
- Aggregation and grouping

The execution time must also be measured and compared with simple processing.

Example (FIFA Dataset): compute a weighted player score:

player score = 0.4*pace + 0.3*shooting + 0.3*dribbling

3. Parallel Processing using Outer Threading:

Students must implement a parallel version using Outer Threading. The implementation should use: **extends Thread** or **implements Runnable**.

Students must divide the dataset among multiple threads and process files in parallel.

The program must be executed using:

- 2 threads
- 4 threads
- 8 threads
- 16 threads
- 32 threads
- 64 threads
- 128 threads

For each execution: record the execution time and analyze the results.

4. Parallel Processing using Inner Threading:

Students must also implement the same processing again using inner threading. They may use: **inner thread**, **inner runnable**, or **lambda**.

The same thread counts must be tested:

- 2 threads
- 4 threads
- 8 threads
- 16 threads
- 32 threads
- 64 threads
- 128 threads

For each execution: record the execution time and analyze the results.

Important Note: Increasing the number of threads does not always improve performance. Excessive threads may reduce efficiency because of scheduling overhead, memory contention, and context switching.

5. Performance Analysis:

Students must compare:

- a) Sequential Execution vs. Parallel Execution
- b) Simple Processing vs. Complex Processing
- c) Different numbers of threads

Submission Requirements:

Each group must submit:

- Java Project
- Final Report (PDF)

Students must submit the source code and report compressed as a single .zip or .rar file.

ALL GROUP MEMBERS ARE EXPECTED TO CONTRIBUTE EQUALLY